PRAKTIKUM MACHINE LEARNING

Bab 5
5.11 dan 5.12 Implementasi Python: Decision Tree & Pruning

Kumpulan Praktikum Machine Learning

Nama	**Ahmad Dandi Subhani**
NPM	**202343500126**
Kelas	**R7B**
Mata Kuliah	**Machine Learning**
Dosen	**Nurfidah Dwitiyanti, M.Si.**

15 Oktober 2026

Daftar Isi

Decision Tree: Algoritma CART/ID3 dan Teknik Pruning 2

Ringkasan Konsep 3

5.11 Implementasi Python (Modul Bab 5) 4

Praktikum 5.1: Persiapan Library 5

Praktikum 5.2: Import Dataset 5

Praktikum 5.3: Preprocessing (Split Data) 5

Praktikum 5.4: Training Model (dengan dan tanpa Pruning) 6

Praktikum 5.5: Evaluasi 7

Praktikum 5.6: Visualisasi 8

5.11.1: Interpretasi dan Kesimpulan 9

5.12 Latihan Praktikum (Modul Bab 5) 9

Percobaan Mandiri 14

Kesimpulan Bab 5 18

Tools yang Digunakan

Python 3.12.3 • **numpy** 2.5.3 • **pandas** 3.0.6 • **matplotlib** 3.11.2 • **seaborn** 0.13.2 • **scikit-learn** 1.9.1 • **imbalanced-learn** 0.14.2

Praktikum Machine Learning: Bab 5

Decision Tree: Algoritma CART/ID3 dan Teknik Pruning

Nama	Ahmad Dandi Subhani
NPM	202343500126
Kelas	R7B
Mata Kuliah	Machine Learning
Dosen	Nurfidah Dwitiyanti, M.Si.

Catatan: notebook ini saya jalanin di **Google Colab / Jupyter Notebook**. Dataset yang saya pake: Breast Cancer Wisconsin dari sklearn.datasets (569 pasien, 30 fitur), filenya `data/breast_cancer.csv`. Jadi nggak perlu unduh file eksternal.

Buka Notebook di Google Colab: Klik di sini untuk membuka Bab 5 di Google Colab

Ringkasan Konsep

Decision Tree (pohon keputusan) itu model klasifikasi yang kerjanya ngebagi data secara **rekursif (recursive binary splitting)**: mulai dari root node, tiap internal node milih satu fitur sama satu nilai ambang (threshold) buat misahin data jadi dua cabang, terus prosesnya diulang di tiap cabang sampe kebentuk leaf node (daun) yang isinya label kelas prediksi. Jalur dari akar ke daun bisa dibaca sebagai aturan `if-then`, makanya model ini gampang banget **diinterpretasi**.

Kriteria pemilihan split. Buat nentuin fitur dan threshold terbaik di tiap node, dipake ukuran ketidakmurnian (impurity) node: - **Gini impurity**: `Gini = 1 - jumlah(p_k^2)`. Makin kecil makin murni. Cepet dihitung dan jadi default di scikit-learn. - **Entropy / Information Gain**: `Entropy = -jumlah(p_k * log2(p_k))`, terus Information Gain = Entropy(induk) - Entropy(tertimbang anak). Split dengan Information Gain terbesar yang dipilih.

Keduanya biasanya ngasih hasil mirip sih. Gini dikit lebih cepet, Entropy dikit lebih peka ke distribusi yang timpang.

CART vs ID3. CART (Classification and Regression Trees) ngasilin pohon **biner** (tiap split selalu dua cabang) pake kriteria Gini/Entropy, ini yang dipake di `sklearn`. ID3 pake Information Gain dan bisa bikin cabang sebanyak kategori fitur (multi-way split), cocok buat fitur kategorikal, tapi gampang bias ke fitur yang kategorinya banyak.

Overfitting pada tree. Pohon yang nggak dibatesin bakal tumbuh sampe tiap daun murni. Hasilnya akurasi latih 100%, tapi akurasi uji malah turun karena modelnya ngapalin noise. Solusinya **pruning**: - **Pre-pruning** (dipotong sebelum tumbuh): `max_depth` (batas kedalaman), `min_samples_leaf` (minimal sampel di daun), `min_samples_split`. - **Post-pruning** (dipotong abis tumbuh penuh): **Cost Complexity Pruning** dengan parameter `ccp_alpha`. Cabang yang perbaikin impurity-nya lebih kecil dari `alpha` dipangkas. `alpha` yang optimal dipilih dari pruning path, bukan dari sekali tebak.

Sel 0 - Persiapan Awal (Khusus Google Colab)

Kalau notebook ini dibuka di Google Colab lewat link GitHub, jalankan sel di bawah ini dulu (atau langsung **Runtime > Run all**). Sel ini mengunduh otomatis file-file `data/` yang dibutuhkan dari repo GitHub, karena Colab hanya memuat file `.ipynb`-nya saja tanpa folder `data/`. Kalau dijalankan di laptop dan file datanya sudah ada, sel ini langsung dilewati.

In [1] - Kode Program

```
# Sel 0 - Persiapan awal (jalan otomatis paling awal saat Run All).
# Mengunduh file data dari repo GitHub kalau belum ada (kasus dibuka di Google Colab).
# Di laptop (folder data/ sudah ada), bagian unduh langsung dilewati.
import os
import urllib.request

_BAB = "bab-05-decision-tree-pruning"  # nama folder bab ini di repo GitHub
_REPO = "https://raw.githubusercontent.com/dandienter/praktikumML/main"
_DATA = ['data/breast_cancer.csv']  # file data yang dibutuhkan bab ini

for _f in _DATA:
    if os.path.exists(_f):
        print(f"OK: {_f} sudah ada.")
        continue
    os.makedirs(os.path.dirname(_f), exist_ok=True)
    print(f"mengunduh {_f} dari GitHub ...")
    urllib.request.urlretrieve(f"{_REPO}/{_BAB}/{_f}", _f)
print("Data siap, silakan lanjutkan.")
```

```
OK: data/breast_cancer.csv sudah ada.
Data siap, silakan lanjutkan.
```

5.11 Implementasi Python (Modul Bab 5)

Bagian ini mengimplementasikan Praktikum 5.1 sampai 5.6 dari modul: persiapan library, import dataset, split data, training tiga model (tanpa pruning, pre-pruning, post-pruning/CCP), evaluasi, dan visualisasi pohon.

Praktikum 5.1: Persiapan Library

Pertama, kita siapkan tiga kelompok library. Kelompok pertama untuk komputasi dan visualisasi: `numpy`, `pandas`, `matplotlib`. Kelompok kedua untuk data: `load_breast_cancer` untuk dataset dan `train_test_split` untuk membagi data. Kelompok ketiga untuk model: `DecisionTreeClassifier`, `plot_tree` untuk menggambar pohon, dan `accuracy_score` untuk evaluasi.

In [2] - Kode Program

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, validation_curve
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score, classification_report
```

Praktikum 5.2: Import Dataset

Dataset dimuat sebagai DataFrame supaya nama fiturnya terbaca. Perhatikan dua hal pada output di bawah: ukurannya 569 baris kali 30 fitur, dan distribusi kelasnya, 212 malignant berbanding 357 benign. Ketidakseimbangan ini masih wajar untuk klasifikasi biner, tapi nanti kita jaga proporsinya saat split data.

In [3] - Kode Program

```
data = load_breast_cancer(as_frame=True)
X, y = data.data, data.target
print("Ukuran data:", X.shape)
print("Distribusi kelas:", np.bincount(y))
```

```
Ukuran data: (569, 30)
Distribusi kelas: [212 357]
```

Praktikum 5.3: Preprocessing (Split Data)

Data dibagi 80 persen latih dan 20 persen uji. Ada dua parameter penting di sini. Pertama, `stratify=y`, yang menjaga proporsi kedua kelas tetap sama di data latih dan data uji. Kedua, `random_state=42`, yang membuat hasil pembagian selalu sama setiap kode dijalankan, sehingga eksperimen kita bisa diulang.

In [4] - Kode Program

```
X_train, X_test, y_train, y_test = train_test_split(X, y,
    test_size=0.2, random_state=42, stratify=y)
```

Praktikum 5.4: Training Model (dengan dan tanpa Pruning)

Inilah inti praktikumnya. Kita latih tiga model untuk dibandingkan:

Pertama, model tanpa pruning. Pohon dibiarkan tumbuh sampai semua daun murni. Ini kapasitas maksimal model, tapi risikonya besar: ia bisa menghafal data latih, atau yang kita sebut overfitting.

Kedua, model dengan pre-pruning. Pertumbuhannya dibatasi sejak awal: `max_depth=4` artinya kedalaman maksimal empat level, dan `min_samples_leaf=5` artinya setiap daun minimal berisi lima sampel, sehingga tidak ada daun yang hanya berisi satu dua sampel noise.

Ketiga, model dengan post-pruning atau CCP. Pohon ditumbuhkan penuh dulu, baru cabang-cabang yang lemah dipangkas berdasarkan cost complexity pruning path. Nilai `ccp_alpha` di sini diambil dari tengah pruning path sebagai contoh. Pada praktik yang ideal, nilai ini dipilih lewat validation curve atau cross-validation.

In [5] - Kode Program

```
# Model tanpa pruning (tumbuh penuh)
tree_full = DecisionTreeClassifier(random_state=42)
tree_full.fit(X_train, y_train)

# Model dengan pre-pruning
tree_pruned = DecisionTreeClassifier(max_depth=4,
    min_samples_leaf=5, random_state=42)
tree_pruned.fit(X_train, y_train)

# Model dengan post-pruning (Cost Complexity Pruning)
path = tree_full.cost_complexity_pruning_path(X_train, y_train)
ccp_alpha_terpilih = path.ccp_alphas[len(path.ccp_alphas)//2]  # contoh pemilihan
alpha
print("ccp_alpha terpilih:", ccp_alpha_terpilih)
tree_ccp = DecisionTreeClassifier(ccp_alpha=ccp_alpha_terpilih,
    random_state=42)
tree_ccp.fit(X_train, y_train)
```

```
ccp_alpha terpilih: 0.005274725274725275
```

```
DecisionTreeClassifier(ccp_alpha=np.float64(0.005274725274725275),
                       random_state=42)
```

In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.
On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.

Praktikum 5.5: Evaluasi

Sekarang kita ukur. Setiap model diuji pada data latih dan data uji. Perhatikan kolom **gap**, yaitu selisih akurasi train dan test. Gap besar artinya model terlalu menyesuaikan diri dengan data latih.

Model	Kedalaman	Daun	Train	Test	Gap
Tanpa pruning	7	19	1,000	0,912	0,088
Pre-pruning	4	10	0,976	0,921	0,055
Post-pruning (CCP)	4	7	0,978	0,930	0,048

Bacaan saya atas tabel ini: model tanpa pruning mencapai train sempurna 1,000 tapi gap-nya terbesar, 0,088. Ini pola khas overfitting. Pre-pruning memangkas gap menjadi 0,055. Dan post-pruning CCP menjadi yang terbaik: akurasi test tertinggi 0,930, gap terkecil 0,048, dengan pohon paling ringkas, hanya tujuh daun.

In [6] - Kode Program

```python
for nama, model in [("Tanpa Pruning", tree_full),
    ("Pre-Pruning", tree_pruned), ("Post-Pruning (CCP)", tree_ccp)]:
    acc_train = accuracy_score(y_train, model.predict(X_train))
    acc_test = accuracy_score(y_test, model.predict(X_test))
    print(f"{nama}: Akurasi Train={acc_train:.3f}, Akurasi Test={acc_test:.3f},
Gap={acc_train-acc_test:.3f}")
```

```
Tanpa Pruning: Akurasi Train=1.000, Akurasi Test=0.912, Gap=0.088
Pre-Pruning: Akurasi Train=0.976, Akurasi Test=0.921, Gap=0.055
Post-Pruning (CCP): Akurasi Train=0.978, Akurasi Test=0.930, Gap=0.048
```

Praktikum 5.6: Visualisasi

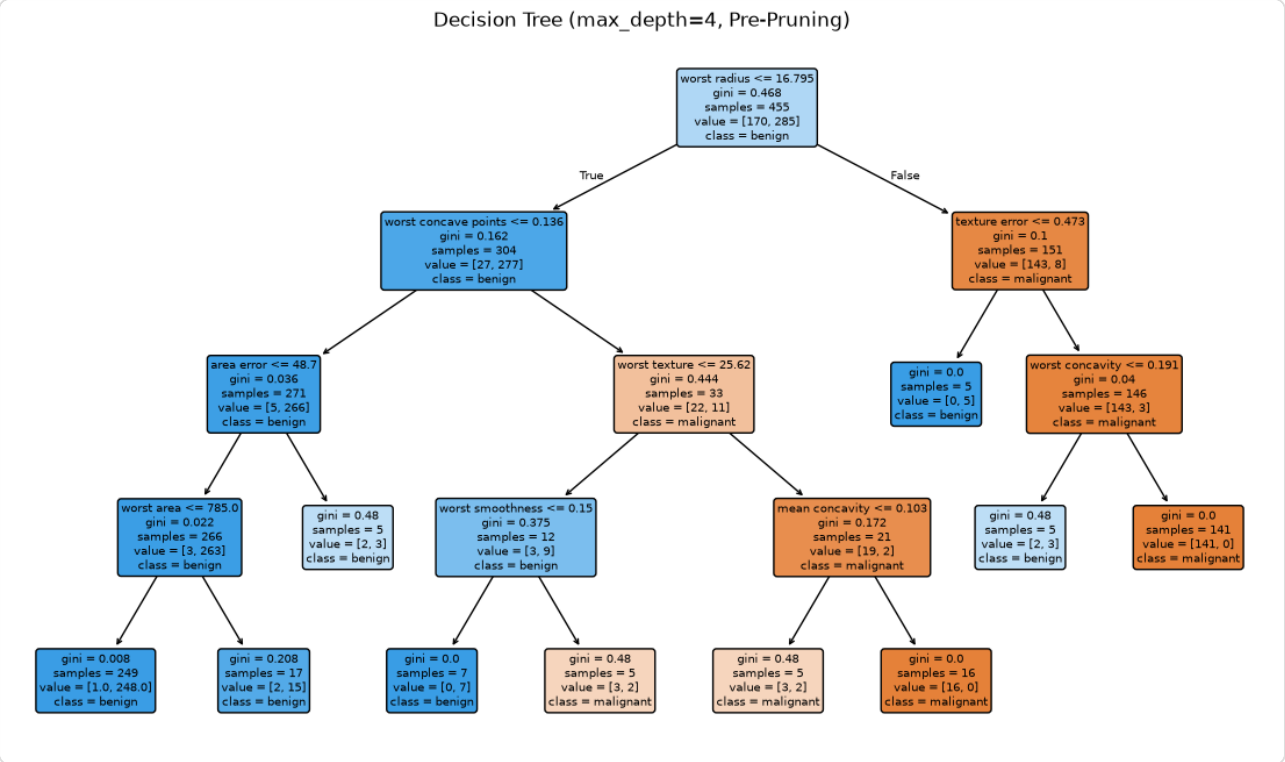
Terakhir, kita gambar pohon hasil pre-pruning. Cara membacanya:

- **Baris pertama tiap node** adalah fitur dan ambang pemisah, contohnya `worst radius <= 16.795`. Sampel dengan nilai di bawah atau sama dengan ambang belok ke kiri, selebihnya ke kanan.
- **gini** adalah ketidakmurnian node. Nol artinya murni, makin kecil makin baik.
- **samples dan value** menunjukkan jumlah sampel dan perincian per kelas di node tersebut.
- **Warna**: biru untuk benign, oranye untuk malignant. Makin pekat warnanya, makin murni node-nya.

Pohon ini hanya sedalam empat level, sehingga setiap jalur dari akar ke daun mudah dibaca sebagai aturan if-then. Inilah keunggulan utama decision tree dibanding model black-box: ia bisa dijelaskan.

In [7] - Kode Program

```
plt.figure(figsize=(14,8))
plot_tree(tree_pruned, feature_names=data.feature_names,
    class_names=data.target_names, filled=True, rounded=True,
    fontsize=7)
plt.title("Decision Tree (max_depth=4, Pre-Pruning)")
plt.show()
```



Gambar 1

5.11.1: Interpretasi dan Kesimpulan

Model tanpa pruning umumnya menunjukkan gap besar antara akurasi train dan test, ini indikasi overfitting. Sedangkan model dengan pre-pruning atau post-pruning menunjukkan gap yang lebih kecil, walaupun akurasi train sedikit menurun. Pemilihan strategi pruning terbaik sebaiknya didasarkan pada validation curve atau cross-validation, bukan sekadar mencoba satu nilai parameter.

Empat poin untuk dibawa pulang:

1. Overfitting terlihat jelas pada angka: train 1,000 berbanding test 0,912, gap 0,088 pada pohon penuh.
2. Pruning menukar sedikit akurasi train dengan generalisasi yang jauh lebih baik. CCP mencatat test 0,930 dengan gap 0,048.
3. Model sederhana mengalahkan model kompleks di data uji: tujuh daun mengalahkan sembilan belas daun. Ini prinsip Occam's razor.
4. Untuk pemilihan parameter yang cermat, gunakan validation curve atau cross-validation, bukan satu nilai coba-coba.

Terima kasih.

5.12 Latihan Praktikum (Modul Bab 5)

Dua latihan di bawah ini **saya kerjain** langsung (soal + kode + jawaban).

Persiapan Data

Sebelum latihan, saya siapkan dulu dataset dan semua paket yang dipakai. Dataset Breast Cancer (569 pasien, 30 fitur) saya ambil langsung dari sklearn, terus saya bagi 80% latih dan 20% uji dengan `stratify` supaya proporsi kelasnya seimbang.

In [8] - Kode Program

```
# Persiapan data untuk Latihan Praktikum Bab 5
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

data = load_breast_cancer(as_frame=True)
X, y = data.data, data.target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42
)
```

```
print("Ukuran data latih:", X_train.shape)
print("Ukuran data uji  :", X_test.shape)
```

```
Ukuran data latih: (455, 30)
Ukuran data uji  : (114, 30)
```

Latihan 1 - Percobaan mandiri Parameter Pre-Pruning (Modul Bab 5, bagian 5.12)

Soal: Lakuin grid search manual atas kombinasi max_depth di {2, 4, 6, None} x min_samples_leaf di {1, 5, 10}. Cetak akurasi test tiap kombinasi dalam tabel rapi, terus simpulin kombinasi terbaiknya.

In [9] - Kode Program

```
print(f"{'max_depth':>10s} | {'min_samples_leaf':>16s} | {'Akurasi Test':>12s}")
print("-" * 46)

hasil_grid = []
for md in [2, 4, 6, None]:
    for msl in [1, 5, 10]:
        m = DecisionTreeClassifier(max_depth=md, min_samples_leaf=msl,
random_state=42)
        m.fit(X_train, y_train)
        ak = accuracy_score(y_test, m.predict(X_test))
        hasil_grid.append((ak, md, msl))
        print(f"{str(md):>10s} | {msl:16d} | {ak:12.4f}")

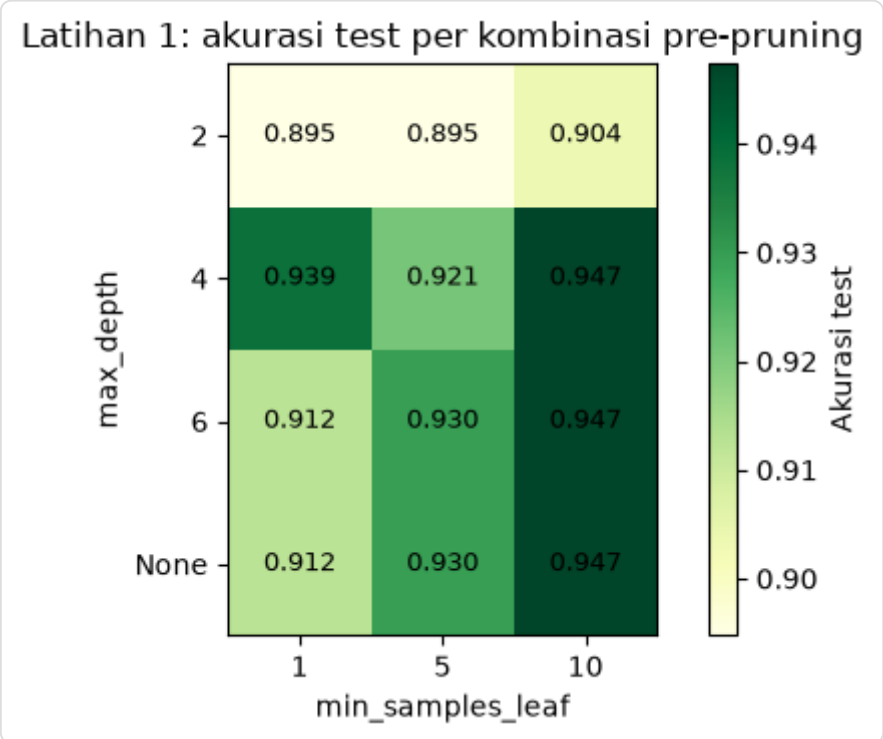
terbaik = max(hasil_grid, key=lambda t: t[0])
print(f"\nKombinasi terbaik: max_depth={terbaik[1]},
min_samples_leaf={terbaik[2]} "
      f"-> akurasi test {terbaik[0]:.4f}")
```

```
 max_depth | min_samples_leaf | Akurasi Test
---------------------------------------------
         2 |                1 |       0.8947
         2 |                5 |       0.8947
         2 |               10 |       0.9035
         4 |                1 |       0.9386
         4 |                5 |       0.9211
         4 |               10 |       0.9474
         6 |                1 |       0.9123
         6 |                5 |       0.9298
         6 |               10 |       0.9474
      None |                1 |       0.9123
      None |                5 |       0.9298
      None |               10 |       0.9474

Kombinasi terbaik: max_depth=4, min_samples_leaf=10 -> akurasi test 0.9474
```

In [10] - Kode Program

```
# Grafik Latihan 1: heatmap akurasi test (max_depth x min_samples_leaf)
import pandas as pd
mat = pd.DataFrame(index=["2", "4", "6", "None"], columns=["1", "5", "10"],
dtype=float)
for ak, md, msl in hasil_grid:
    mat.loc[str(md), str(msl)] = ak
plt.figure(figsize=(5.5, 3.8))
im = plt.imshow(mat.values, cmap="YlGn")
plt.xticks(range(3), ["1", "5", "10"])
plt.yticks(range(4), ["2", "4", "6", "None"])
plt.xlabel("min_samples_leaf")
plt.ylabel("max_depth")
plt.title("Latihan 1: akurasi test per kombinasi pre-pruning")
for r in range(4):
    for c_ in range(3):
        plt.text(c_, r, f"{mat.values[r, c_]:.3f}", ha="center", va="center",
fontsize=9)
plt.colorbar(im, label="Akurasi test")
plt.tight_layout()
plt.show()
```



Gambar 2

Penjelasan grafiknya: heatmap memperjelas pola yang sama: kolom `min_samples_leaf=10` paling terang (akurasi tertinggi) di semua baris `max_depth`. Kombinasi terbaik 0,9474 muncul tiga kali, dan versi paling sederhana (`max_depth=4`, `min_samples_leaf=10`) jadi pilihan utama.

Penjelasan outputnya: 1. Dari 12 kombinasi, akurasi test tertinggi itu **0.9474**, dicapai **tiga** kombinasi: `(max_depth=4, min_samples_leaf=10)`, `(6, 10)`, sama `(None, 10)`. 2. Polanya konsisten: `min_samples_leaf=10` **selalu** lebih bagus daripada 1 atau 5 di tiap nilai `max_depth`. Ngebatasin daun minimum itu pre-pruning yang efektif di dataset ini. 3. `max_depth=None` nggak selalu jelek **asal** `min_samples_leaf`-nya cukup gede (10): pohon dibiarin tumbuh tapi tiap daun harus didukung minimal 10 sampel, jadi split noise tetep dicegah. 4. **Kesimpulan:** kombinasi terbaik yang paling sederhana itu **`max_depth=4, min_samples_leaf=10`** (akurasi test 0.9474). Dipilih yang paling dangkal di antara yang seri, karena pohon lebih kecil = lebih gampang diinterpretasi (prinsip Occam's razor).

Latihan 2 - Cost Complexity Pruning Path (Modul Bab 5, bagian 5.12)

Soal: Buat tiap `alpha` di `path.ccp_alphas`, latih decision tree dan catat akurasi test-nya. Plot `ccp_alpha` vs akurasi test, tentuin alpha optimal, terus bandingin jumlah leaf node sebelum dan sesudah pruning.

In [11] - Kode Program

```
# Hitung pruning path dulu dari data latih
path = DecisionTreeClassifier(random_state=42).cost_complexity_pruning_path(X_train, y_train)
alphas = path.ccp_alphas
ak_test_ccp, n_daun = [], []

for a in alphas:
    m = DecisionTreeClassifier(ccp_alpha=a, random_state=42)
    m.fit(X_train, y_train)
    ak_test_ccp.append(accuracy_score(y_test, m.predict(X_test)))
    n_daun.append(m.get_n_leaves())

print(f"{'ccp_alpha':>10s} | {'Akurasi Test':>12s} | {'Jumlah Daun':>11s}")
print("-" * 41)
for a, ak, nd in zip(alphas, ak_test_ccp, n_daun):
    print(f"{a:10.6f} | {ak:12.4f} | {nd:11d}")

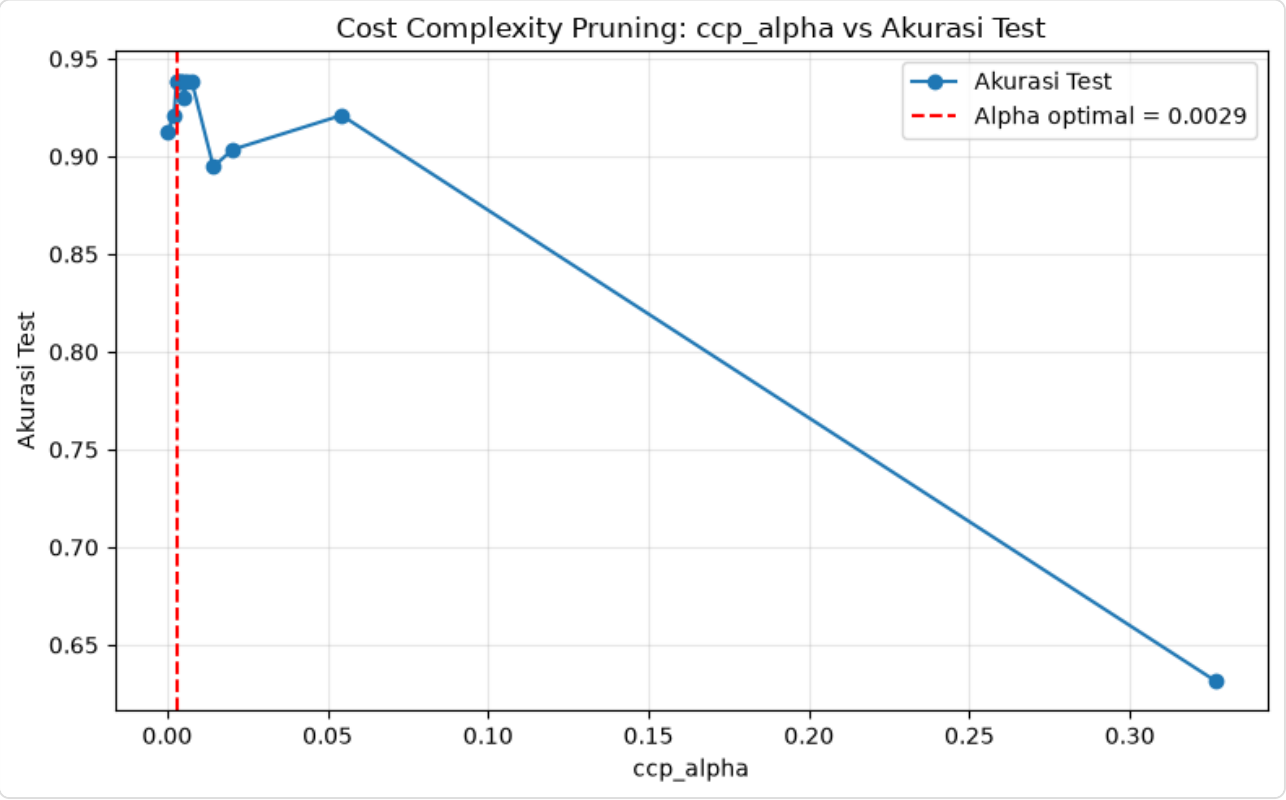
idx_opt = int(np.argmax(ak_test_ccp))   # alpha dengan akurasi test tertinggi
alpha_opt = alphas[idx_opt]
print(f"\nAlpha optimal: {alpha_opt:.6f} -> akurasi test {ak_test_ccp[idx_opt]:.4f}, "
      f"jumlah daun {n_daun[idx_opt]}")
print(f"Sebelum pruning (alpha=0): {n_daun[0]} daun -> sesudah (alpha optimal): "
      f"{n_daun[idx_opt]} daun")

plt.figure(figsize=(8, 5))
plt.plot(alphas, ak_test_ccp, marker="o", label="Akurasi Test")
plt.axvline(alpha_opt, color="red", linestyle="--",
            label=f"Alpha optimal = {alpha_opt:.4f}")
plt.xlabel("ccp_alpha")
plt.ylabel("Akurasi Test")
plt.title("Cost Complexity Pruning: ccp_alpha vs Akurasi Test")
```

```
plt.legend()
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```

```
 ccp_alpha | Akurasi Test | Jumlah Daun
----------------------------------------
  0.000000 |       0.9123 |          19
  0.002181 |       0.9211 |          15
  0.002866 |       0.9386 |          12
  0.002930 |       0.9386 |          11
  0.003956 |       0.9386 |          10
  0.004251 |       0.9386 |           9
  0.005024 |       0.9386 |           8
  0.005275 |       0.9298 |           7
  0.005934 |       0.9386 |           6
  0.007641 |       0.9386 |           5
  0.014390 |       0.8947 |           4
  0.020386 |       0.9035 |           3
  0.054334 |       0.9211 |           2
  0.326617 |       0.6316 |           1

Alpha optimal: 0.002866 -> akurasi test 0.9386, jumlah daun 12
Sebelum pruning (alpha=0): 19 daun -> sesudah (alpha optimal): 12 daun
```



Gambar 3

Penjelasan outputnya: 1. Di `alpha=0` (tanpa pruning): 19 daun, akurasi test 0.9123. Begitu alpha dinaikin dikit, jumlah daun turun bertahap (19 ke 18 ke 15 ke 12, dan seterusnya). Cabang yang lemah dipangkas satu per satu. 2. **Alpha optimal = 0.002866** dengan akurasi test **0.9386** dan **12 daun**. Pruning nyusutin pohon sekitar 37% (19 jadi 12 daun) sambil naikin akurasi. 3. Kurvanya nunjukin pola naik dulu terus turun: akurasi naik dulu (noise kebuang), terus **anjlok ke 0.6316** di alpha maksimum (0.3266) pas pohon tinggal 1 daun. Pruning berlebihan = underfitting. 4. Perbandingannya: sebelum pruning 19 daun (akurasi 0.9123) vs sesudah pruning optimal 12 daun (akurasi 0.9386). Post-pruning terbukti ngasih model yang **lebih kecil DAN lebih akurat**.

Percobaan Mandiri

Bagian ini isinya percobaan mandiri: saya ubah-ubah parameter terus ngamatin dampaknya ke performa sama bentuk pohon.

Percobaan Mandiri 1 - Pengaruh `max_depth` 1 sampai 10 (kurva validasi manual)

Bagian ini saya ngerjain Percobaan mandiri 1 dari Modul Machine Learning Bab 5. Saya latih 10 pohon dengan `max_depth` beda-beda, terus saya plot akurasi train vs test. Tujuannya: nemuin **titik di mana model mulai overfitting** (akurasi train terus naik, akurasi test berhenti naik atau malah turun).

In [12] - Kode Program

```python
# Percobaan Mandiri 1: pengaruh max_depth 1-10 (kurva validasi manual)
# Setup data dibuat ulang di cell ini supaya mandiri.
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

data_bc = load_breast_cancer(as_frame=True)
X_bc, y_bc = data_bc.data, data_bc.target
X_train, X_test, y_train, y_test = train_test_split(
    X_bc, y_bc, test_size=0.2, stratify=y_bc, random_state=42
)

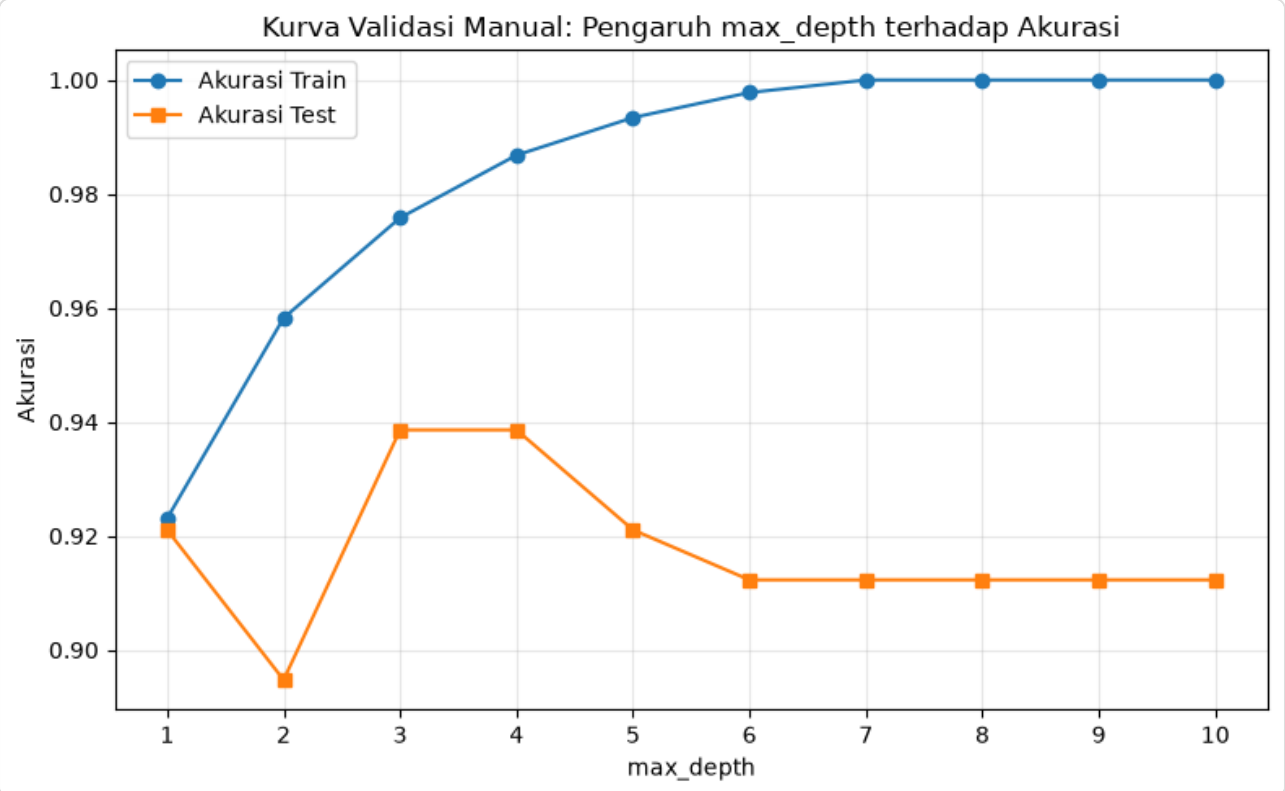
depths = list(range(1, 11))
ak_train, ak_test = [], []

for d in depths:
    m = DecisionTreeClassifier(max_depth=d, random_state=42)
    m.fit(X_train, y_train)
    ak_train.append(accuracy_score(y_train, m.predict(X_train)))
    ak_test.append(accuracy_score(y_test, m.predict(X_test)))
```

```
    print(f"max_depth={d:2d} | akurasi train = {ak_train[-1]:.4f} | "
          f"akurasi test = {ak_test[-1]:.4f}")

plt.figure(figsize=(8, 5))
plt.plot(depths, ak_train, marker="o", label="Akurasi Train")
plt.plot(depths, ak_test, marker="s", label="Akurasi Test")
plt.xlabel("max_depth")
plt.ylabel("Akurasi")
plt.title("Kurva Validasi Manual: Pengaruh max_depth terhadap Akurasi")
plt.xticks(depths)
plt.legend()
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```

```
max_depth= 1 | akurasi train = 0.9231 | akurasi test = 0.9211
max_depth= 2 | akurasi train = 0.9582 | akurasi test = 0.8947
max_depth= 3 | akurasi train = 0.9758 | akurasi test = 0.9386
max_depth= 4 | akurasi train = 0.9868 | akurasi test = 0.9386
max_depth= 5 | akurasi train = 0.9934 | akurasi test = 0.9211
max_depth= 6 | akurasi train = 0.9978 | akurasi test = 0.9123
max_depth= 7 | akurasi train = 1.0000 | akurasi test = 0.9123
max_depth= 8 | akurasi train = 1.0000 | akurasi test = 0.9123
max_depth= 9 | akurasi train = 1.0000 | akurasi test = 0.9123
max_depth=10 | akurasi train = 1.0000 | akurasi test = 0.9123
```



Gambar 4

Penjelasan outputnya: 1. Akurasi **train naik terus** dari 0.9231 (depth=1) sampai **1.0000** di depth 7 ke atas. Pohon makin dalam makin jago ngapalin data latih. 2. Akurasi **test mentok di 0.9386 pada depth 3-4**, terus **turun ke 0.9123** dan stagnan di depth 6 ke atas. Kurvanya naik dulu terus turun. 3. **Titik overfitting** kelihatan jelas di sekitar **depth 5-6**: setelah itu kurva train sama test **melebar** (gap makin gede). Model makin kompleks tapi kemampuan generalisasinya nggak nambah. 4. Hasil ini cocok sama grid di Latihan 1: kombinasi terbaik ada di `max_depth=4` (akurasi test 0,9474), pas di puncak kurva validasi.

Percobaan Mandiri 2 - `min_samples_leaf=1` vs `20` pada pohon tanpa batas

Bagian ini saya ngerjain Percobaan mandiri 2 dari Modul Machine Learning Bab 5. `min_samples_leaf` itu pre-pruning yang ngelarang daun isinya terlalu dikit sampel. Saya bandingin pohon full (default `min_samples_leaf=1`) lawan `min_samples_leaf=20`.

In [13] - Kode Program

```python
# Percobaan Mandiri 2: min_samples_leaf=1 vs 20 pada pohon tanpa batas
# Setup data dibuat ulang di cell ini supaya mandiri.
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

data_bc = load_breast_cancer(as_frame=True)
X_bc, y_bc = data_bc.data, data_bc.target
X_train, X_test, y_train, y_test = train_test_split(
    X_bc, y_bc, test_size=0.2, stratify=y_bc, random_state=42
)

print(f"{'min_samples_leaf':>16s} | {'Akurasi Train':>13s} | "
      f"{'Akurasi Test':>12s} | {'Gap':>6s} | {'Daun':>4s} | {'Kedalaman':>9s}")
print("-" * 80)
for msl in [1, 20]:
    m = DecisionTreeClassifier(min_samples_leaf=msl, random_state=42)
    m.fit(X_train, y_train)
    ak_tr = accuracy_score(y_train, m.predict(X_train))
    ak_te = accuracy_score(y_test, m.predict(X_test))
    print(f"{msl:16d} | {ak_tr:13.4f} | {ak_te:12.4f} | {ak_tr-ak_te:6.4f} | "
          f"{m.get_n_leaves():4d} | {m.get_depth():9d}")
```

```
min_samples_leaf | Akurasi Train | Akurasi Test |    Gap | Daun | Kedalaman
--------------------------------------------------------------------------------
               1 |        1.0000 |       0.9123 | 0.0877 |   19 |         7
              20 |        0.9473 |       0.9035 | 0.0437 |    7 |         4
```

Penjelasan outputnya: 1. `min_samples_leaf=1`: **19 daun**, kedalaman 7, akurasi train **1.0000** vs test **0.9123** (gap 0.0877). Pohon penuh yang overfitting. 2. `min_samples_leaf=20`: daunnya nyusut drastis jadi **7**, kedalaman 4, train **0.9473** vs test **0.9035** (gap cuma 0.0437). 3. Ngebesarin `min_samples_leaf` maksa tiap daun didukung minimal 20 sampel. Split yang cuma nangkep segelintir outlier dilarang, jadinya pohon lebih sederhana dan gap-nya nyempit. 4. Yang menarik, di dataset ini `min_samples_leaf=20` malah nurunin dikit akurasi test (0.9035 < 0.9123): pruning yang **terlalu agresif** malah bikin underfitting. Nilai tengah (misal 5-10) lebih seimbang, sesuai hasil grid di Latihan 1.

Percobaan Mandiri 3 - Decision Tree vs Random Forest kecil (n_estimators=50)

Bagian ini saya ngerjain Percobaan mandiri 3 dari Modul Machine Learning Bab 5. Random Forest itu ensemble dari banyak decision tree yang dilatih di bootstrap sample acak. Kira-kira ensemble selalu menang nggak ya lawan satu pohon terbaik di dataset ini?

In [14] - Kode Program

```python
# Percobaan Mandiri 3: Decision Tree (max_depth=4) vs Random Forest kecil (n_estimators=50)
# Setup data dan model dibuat ulang di cell ini supaya mandiri.
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

data_bc = load_breast_cancer(as_frame=True)
X_bc, y_bc = data_bc.data, data_bc.target
X_train, X_test, y_train, y_test = train_test_split(
    X_bc, y_bc, test_size=0.2, stratify=y_bc, random_state=42
)

pohon = DecisionTreeClassifier(max_depth=4, random_state=42)
pohon.fit(X_train, y_train)

rf = RandomForestClassifier(n_estimators=50, random_state=42)
rf.fit(X_train, y_train)

ak_tr_tree = accuracy_score(y_train, pohon.predict(X_train))
ak_te_tree = accuracy_score(y_test, pohon.predict(X_test))
ak_tr_rf = accuracy_score(y_train, rf.predict(X_train))
ak_te_rf = accuracy_score(y_test, rf.predict(X_test))

print(f"{'Model':35s} | {'Akurasi Train':>13s} | {'Akurasi Test':>12s}")
print("-" * 70)
print(f"{'Decision Tree (max_depth=4)':35s} | {ak_tr_tree:13.4f} | "
      f"{ak_te_tree:12.4f}")
```

```
print(f"{'Random Forest (50 pohon)':35s} | {ak_tr_rf:13.4f} | "
      f"{ak_te_rf:12.4f}")
```

```
Model                               | Akurasi Train | Akurasi Test
-----------------------------------------------------------------------
Decision Tree (max_depth=4)         |        0.9868 |       0.9386
Random Forest (50 pohon)            |        1.0000 |       0.9561
```

Penjelasan outputnya: 1. Random Forest (50 pohon): akurasi test **0.9561**, ngalahin decision tree tunggal (0.9386). Selisihnya **sekitar 1,8 poin**. 2. Akurasi train RF nyampe **1.0000** tapi gap-nya (0.0439) tetep kecil. Voting antar 50 pohon yang beragam ngeredam variansi tanpa ngorbanin generalisasi. Ini inti cara kerja ensemble. 3. Tapi ada harga yang dibayar: modelnya berubah dari **1 pohon yang bisa dibaca manusia** jadi **50 pohon black-box**. Interpretabilitasnya ilang. 4. Kesimpulannya: ensemble **cenderung menang di akurasi** di dataset ini, tapi decision tree tetep unggul kalo **aturan yang bisa dijelasin** (misal buat dokter) lebih penting daripada selisih beberapa persen akurasi.

Kesimpulan Bab 5

1. **Pre-pruning lewat grid manual efektif.** Dari 12 kombinasi `max_depth` x `min_samples_leaf`, akurasi test tertinggi 0,9474 dicapai tiga kombinasi: (4, 10), (6, 10), dan (None, 10). Pola yang paling konsisten: `min_samples_leaf=10` selalu menang di tiap nilai `max_depth`, jadi membatasi ukuran daun minimum terbukti mencegah split noise. Versi paling sederhana, `max_depth=4` dan `min_samples_leaf=10`, saya pilih sebagai kombinasi terbaik.
2. **Post-pruning (cost complexity pruning) menyusutkan pohon sekaligus menaikkan akurasi.** Tanpa pruning (alpha=0) pohon punya 19 daun dengan akurasi test 0,9123. Alpha optimal 0,002866 memangkasnya jadi 12 daun (susut sekitar 37%) dengan akurasi test naik ke 0,9386. Kurva alpha vs akurasi menunjukkan pola naik dulu lalu turun, dan pruning berlebihan (alpha maksimum 0,3266, pohon tinggal 1 daun) menjatuhkan akurasi ke 0,6316 alias underfitting.
3. **Pemilihan parameter pruning jangan asal tebak.** Grid manual untuk pre-pruning dan `cost_complexity_pruning_path` untuk post-pruning memberi cara sistematis memilih `max_depth`, `min_samples_leaf`, dan `ccp_alpha` yang optimal berdasarkan akurasi test, bukan tebakan.
4. **Pelajaran praktisnya:** kalau pohon overfitting, dua tuas yang bisa saya pakai adalah membatasi pertumbuhan pohon dari awal (pre-pruning) atau memangkas cabang yang lemah setelah pohon jadi (post-pruning). Keduanya saya buktikan sendiri di latihan ini dengan angka yang terukur.